

---

## NumPy の基礎

多次元配列とベクトル化された計算

```
import numpy as np
```

# この資料の流れ

---

NumPy とは

配列の作成と属性

インデックスとスライス

演算とブロードキャスト

行列の積とニューラルネットワーク

---

## NumPy とは

## なぜ NumPy か

---

- NumPy は 多次元配列 (ndarray) を効率よく扱うためのライブラリ
- ループを書かずに、配列全体へまとめて演算できる (ベクトル化)
- 数値計算が高速で、機械学習やデータ分析の土台になる

```
# Python のリストは要素ごとにループが必要
xs = [1, 2, 3]
ys = [x * 2 for x in xs] # [2, 4, 6]

# NumPy なら配列全体に一度で演算できる
import numpy as np
a = np.array([1, 2, 3])
a * 2 # array([2, 4, 6])
```

## ndarray のイメージ

---

- 1次元：ベクトル    `shape (4,)`
- 2次元：行列    `shape (3, 2)`
- N次元へ一般化できる

横の並びを**行 (row)**、縦の並びを**列 (column)**と呼ぶ。

1次元 (4,)

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
|---|---|---|---|

2次元 (3,2)

|  |  |
|--|--|
|  |  |
|  |  |
|  |  |



## 配列の作成と属性

## 配列を作る

---

```
np.array([1, 2, 3]) # リストから [1 2 3]
np.zeros((2, 3)) # 全要素 0 の 2x3
np.ones((2, 3)) # 全要素 1 の 2x3
np.arange(0, 10, 2) # 0,2,4,6,8 (間隔指定)
np.linspace(0, 1, 5) # 0~1 を5等分
np.arange(6).reshape(2, 3) # 形を (2,3) に変える
```

### reshape のポイント

要素数が一致する形にのみ変形できる。

6 要素 → (2,3) や (3,2) は可、(2,4) は不可。

## 配列の属性

---

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])  
A.shape # (2, 3) 形状 (行, 列)  
A.ndim # 2 次元数  
A.size # 6 要素の総数  
A.dtype # int64 要素の型  
A.astype(np.float64) # 型を変換
```

- shape はタプルで返る (1 次元でも (4, ) のように)
- 計算前に shape を確認する習慣が、形状エラーを防ぐ





## インデックスとスライス

## スライスとインデックス

---

```
a = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]
a[2] # 2
a[2:5] # [2 3 4] (開始:終了)
a[::2] # [0 2 4 6 8] (1つおき)

A = np.array([[1, 2, 3],
              [4, 5, 6]])
A[0, 1] # 2 (行0, 列1)
A[1] # [4 5 6] (1行目)
A[:, 0] # [1 4] (0列目すべて)
```

2次元では A[行, 列] の形で指定する。

## 条件（bool）インデックス

---

```
x = np.array([-1.0, 1.0, 2.0])  
x > 0 # [False True True]  
x[x > 0] # [1. 2.] 条件を満たす要素だけ  
  
# 条件で値を置き換える  
x[x < 0] = 0 # x -> [0. 1. 2.]
```

### 活性化関数とのつながり

ステップ関数 `np.array(x > 0, dtype=np.int_)` は、この「条件 → True/False → 0/1」の仕組みそのもの。

---

## 演算とブロードキャスト

## 要素ごとの演算

---

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
a + b # [11 22 33]
a * b # [10 40 90]
a + 100 # [101 102 103] スカラーにも
```

- 同じ形どうしの + - \* / は要素ごとに行われる
- スカラーとの演算は、全要素に適用される
- 形が違ってても、規則に従って自動的にそろえられる → ブロードキャスト

## ブロードキャスト

---

形の小さい配列が、大きい配列に合わせて自動的に拡張される。

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

$$\text{shape } (2, 3) + (3,) \rightarrow (2, 3)$$

- $(3,)$  の行が、各行にコピーされて足される
- ループを書かずに、行ごとのバイアス加算などが書ける

## ユニバーサル関数 (ufunc)

---

配列の各要素に対して、関数をまとめて適用する。

```
x = np.array([-1.0, 0.0, 2.0])  
np.exp(x) # 各要素に  $e^x$   
np.maximum(0, x) # [0. 0. 2.] 要素ごとに大きい方  
1 / (1 + np.exp(-x)) # シグモイドも一行で
```

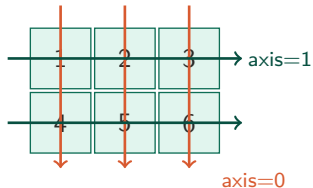
### 活性化関数が一行で書ける理由

`np.exp` や `np.maximum` が配列対応 (ufunc) だから、  
シグモイドや ReLU をループなしで実装できる。

## 集約と axis

```
A = np.array([[1, 2, 3],
               [4, 5, 6]])
A.sum() # 21 全体
A.sum(axis=0) # [5 7 9] 列ごと
A.sum(axis=1) # [6 15] 行ごと
A.mean(), A.max()
```

axis=0 は縦（行をまたいで集約）、  
axis=1 は横（列をまたいで集約）。





---

## 行列の積とニューラルネットワーク

## 行列の積

---

\* は要素ごとの積。行列の積は `np.dot` または `@`。

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
np.dot(A, B) # = A @ B
# array([[19, 22],
#        [43, 50]])
```

### 要素ごとの積との違い

`A * B` は同じ位置どうしを掛けるだけ。

`A @ B` は「行 × 列」を掛けて足す行列の積。

## 形状の一致ルール

対応する次元の要素数を一致させる

$$\underbrace{A}_{m \times n} \cdot \underbrace{B}_{n \times p} = \underbrace{C}_{m \times p}$$

- 内側の  $n$  (左の列数 = 右の行数) を一致させる
- 結果は外側の  $m \times p$
- 一致しないと `ValueError (shapes not aligned)`

## ニューラルネットワークへの応用

---

入力  $X$  と重み  $W$  の積で、層の計算がまとめて書ける。

```
X = np.array([1, 2]) # 入力 shape (2,)
W = np.array([[1, 3, 5],
              [2, 4, 6]]) # 重み shape (2, 3)
B = np.array([10, 20, 30]) # バイアス shape (3,)

A = np.dot(X, W) + B # 総和 + バイアス
# A = [15, 31, 47]
Z = 1 / (1 + np.exp(-A)) # 活性化 (シグモイド)
```

- $X(2,)$  と  $W(2,3)$  で形状が一致  $\rightarrow$  出力  $(3,)$
- ブロードキャストでバイアスを加算、ufunc で活性化

## まとめ

---

- NumPy の中心は `ndarray`。 `shape` ・ `ndim` ・ `dtype` を押さえる
- `+` `-` `*` `/` は要素ごと、形が違えばブロードキャストでそろえる
- `np.exp` ・ `np.maximum` などの `ufunc` で活性化関数が一行で書ける
- `sum/mean/max` は `axis` で集約方向を指定する
- 行列の積は `np.dot` / `@`。対応する次元を一致させる

これらが、ニューラルネットワークの計算 ( $XW + B \rightarrow h(\cdot)$ ) の土台になる。